

DomoVida — Artefactos Scrum

Fichas de sprint y retrospectivas — Sprints 1, 2 y 3 (actualizado al 6 de octubre de 2026)

Andrés Rodrigo Vergara Acevedo · Capstone PTY4614 · Duoc UC, sede Viña del Mar

Nota sobre el origen de este registro

Este documento fue **reconstruido el 1 de octubre de 2026 a partir del historial de commits del repositorio** <https://github.com/AndresVergara74/domovida-backend> (85 commits entre el 21 de agosto y el 1 de octubre de 2026). Durante los sprints no se mantuvo una bitácora: la herramienta prevista (Obsidian) no pudo usarse con la cuenta institucional. Por eso cada afirmación se respalda con el identificador del commit correspondiente, que registra la fecha real del avance y puede verificarse en GitHub.

Formato según la Guía de apoyo del estudiante (secciones 5.5 Ficha de sprint o ciclo y 5.6 Retrospectiva). Los identificadores de historias (HU-01 a HU-12) corresponden al Product Backlog de la Guía 1.5 (Tabla 10).

Calendario de sprints

La asignatura comenzó el martes 11 de agosto de 2026 (semana S1). El estudiante se incorporó el 19 de agosto. Cada sprint dura tres semanas, según la Tabla 4 de la Guía 1.5.

Tabla A. Calendario de sprints del proyecto DomoVida

Sprint	Semanas	Fechas	Estado al 06/10/2026
Sprint 1	S1-S3	11 ago – 30 ago	Cerrado
Sprint 2	S4-S6	31 ago – 20 sep	Cerrado
Sprint 3	S7-S9	21 sep – 11 oct	En curso (cierra el 11/10; corte al 06/10)
Sprint 4	S10-S12	12 oct – 1 nov	Planificado
Sprint 5	S13-S15	2 nov – 22 nov	Planificado
Sprint 6	S16-S18	23 nov – 13 dic	Planificado

Desviación respecto del plan. El avance real va adelantado: el OE1, planificado para el Sprint 3, se completó en el Sprint 2, y la mayor parte del OE2 y del OE3, planificados para el Sprint 4, se completó en el Sprint 3. Las fichas registran lo que ocurrió y no lo que estaba planificado.

Sprint 1 (S1-S3)

Ficha de sprint

Tabla B. Ficha del Sprint 1

Campo	Contenido
Sprint o ciclo	Sprint 1 · 11 de agosto – 30 de agosto de 2026 (incorporación del estudiante: 19 de agosto)
Objetivo	Definir el proyecto y preparar la base técnica: repositorio Git en GitHub y estructura inicial de carpetas.
Historias comprometidas	Ninguna historia de producto. Tareas de preparación: creación del repositorio, estructura inicial y primeras evidencias de la Fase 1.
Historias terminadas	No aplica (sprint de preparación).
Funcionalidades implementadas	• Repositorio domovida-backend creado en GitHub. • Estructura inicial del proyecto y archivos base.
Pruebas realizadas	No aplica: aún no había código funcional que probar.
Evidencias	• Commit bad5f37 (21/08): «feat: inicializacion proyecto domovida-backend». • Commit d303152 (21/08): «chore: remover desktop.ini del seguimiento». • Commit d15409b (25/08): «agregar archivos».
Problemas encontrados	• Incorporación tardía: el estudiante comenzó el 19/08, una semana después del inicio de la asignatura. • Se subió por error al repositorio un archivo de sistema de Windows (desktop.ini).
Acciones tomadas	• Se creó el repositorio dos días después de la incorporación para no acumular más atraso. • Se retiró desktop.ini del seguimiento de Git (commit d303152).
Pendientes	Base del frontend (React) y del backend (FastAPI) → Sprint 2.

Retrospectiva del Sprint 1

Campo	Contenido
¿Qué funcionó bien?	Crear el repositorio apenas iniciado el trabajo permitió que todo el avance posterior quedara registrado con fecha.
¿Qué problemas tuvimos?	Inicio una semana más tarde que el resto del curso; carga académica y laboral que limita las horas disponibles; mensajes de commit poco descriptivos («agregar archivos»).
¿Qué vamos a cambiar?	Escribir mensajes de commit descriptivos y dejar lista la base técnica antes de empezar el desarrollo funcional.
Acción concreta, responsable y fecha	Crear la base del frontend (Vite + React) y del backend (FastAPI) · Andrés Vergara · antes del 1 de septiembre de 2026.

Sprint 2 (S4-S6)

Ficha de sprint

Tabla C. Ficha del Sprint 2

Campo	Contenido
Sprint o ciclo	Sprint 2 · 31 de agosto – 20 de septiembre de 2026
Objetivo	Contar con la base del frontend y del backend, cerrar las evidencias de la Fase 1 y completar la captura de datos de los 8 sensores simulados (OE1).
Historias comprometidas	HU-01, HU-04 y HU-05 (captura y detección local de eventos).
Historias terminadas	Captura y visualización de los 8 sensores en el panel (base de HU-01, HU-04 y HU-05). Las alertas en tiempo real de HU-01 se completan en el Sprint 3.
Funcionalidades implementadas	• Frontend con Vite + React y backend con FastAPI. • Simulador de 8 sensores IoT enviando datos al backend. • Panel mostrando las lecturas de los sensores. • 8 fichas técnicas de documentación (arquitectura, backend, frontend, sensores, base de datos, panel, alarmas y comandos).
Pruebas realizadas	Verificación funcional manual de los 8 sensores en el panel. No se registraron casos de prueba formales en este sprint.
Evidencias	• Commits 809e7b3 y 7aa817c (01/09): frontend con Vite + React y backend con FastAPI. • Commits 74cca61 y 0141067 (15/09): evidencias de la Fase 1. • Commit 5531323 (18/09): planilla de evaluación de la Fase 1. • Commit 02f5d08 (18/09): «OE1 completo: 8 sensores funcionando con dashboard en tiempo real». • Commits 3a08479 a 47a8301 (18-19/09): fichas técnicas 1 a 8.
Problemas encontrados	• Desorden en los archivos de evidencias: nombres con doble extensión (.docx.docx), carpetas eliminadas y varios renombres el 15/09. • Conflictos de fusión en Git al trabajar desde más de un lugar (commits 3dd81ab y 4334f27). • El botón de pánico fijo (boton_panico_sala) solo cubre el interior del hogar; las caídas o emergencias en el patio, el jardín o fuera de la casa no se detectan.
Acciones tomadas	• Reorganización de las evidencias en carpetas por fase (evidencias_individuales_fase1, evidencias_grupales_fase1). • Resolución de los conflictos de fusión (commit 3dd81ab). • Se definió una ruta de botón de pánico portátil en tres etapas, compatible con la arquitectura actual: (1) el prototipo ya incluye un sensor wearable_cardiaco simulado; (2) a mediano plazo, un reloj de hardware abierto (Bangle.js 2 o PineTime, IP67, con acelerómetro y pulso) conectado por Bluetooth al nodo Edge del hogar, que cubre patio y jardín sin que el adulto mayor deba configurar nada; (3) a largo plazo, una pulsera propia con ESP32 y

Campo	Contenido
	conectividad 4G/GPS para emergencias fuera de casa. Los dispositivos comerciales evaluados en Chile no detectan caídas (botones simples de \$11.000–\$18.000) o cuestan cerca de \$240.000 más una renovación anual (pulseras con GPS y 4G). Se prefiere el desarrollo propio por su menor costo unitario a escala, su integración directa con la plataforma y el cumplimiento de la Ley N° 21.719, validando primero con hardware abierto existente. • En el simulador se ajustó la probabilidad de activación del botón fijo al 2% (commit 3b6edd9).
Pendientes	Notificaciones push, base de datos en la nube, alertas en tiempo real y despliegue → Sprint 3.

Retrospectiva del Sprint 2

Campo	Contenido
Verificación de la acción anterior	Acción del Sprint 1: crear la base del frontend y del backend antes del 1 de septiembre. Se cumplió: ambos se subieron el 01/09 (commits 809e7b3 y 7aa817c). Efecto: permitió completar el OE1 dentro del mismo sprint.
¿Qué funcionó bien?	Se completó el OE1 antes de lo planificado y se documentó en 8 fichas técnicas.
¿Qué problemas tuvimos?	Tiempo perdido en ordenar archivos de evidencias y resolver conflictos de Git.
¿Qué vamos a cambiar?	Mantener una estructura de carpetas fija por fase y hacer commits pequeños con un prefijo que indique el tipo de cambio (feat, fix, docs).
Acción concreta, responsable y fecha	Usar mensajes de commit con prefijo feat/fix/docs en todos los commits del Sprint 3 · Andrés Vergara · desde el 21 de septiembre de 2026.

Sprint 3 (S7-S9) — en curso

Ficha de sprint (corte al 1 de octubre de 2026)

Tabla D. Ficha del Sprint 3

Campo	Contenido
Sprint o ciclo	Sprint 3 · 21 de septiembre – 11 de octubre de 2026 · ficha con corte al 1 de octubre
Objetivo	Llevar DomoVida a la nube con alertas en tiempo real (OE2 y OE3) y desplegarlo en servicios accesibles por URL pública.
Historias comprometidas	HU-01, HU-02, HU-03, HU-06, HU-07, HU-08, HU-09, HU-10 y HU-11.
Historias terminadas	HU-01, HU-02, HU-03, HU-06, HU-07, HU-09, HU-10 y HU-11. En progreso: HU-08 (ver Pendientes).
Funcionalidades implementadas	• Notificaciones push mediante ntfy, con tópico seudonimizado (HU-02). • Base de datos en Supabase/PostgreSQL con Session Pooler IPv4 y triggers PL/pgSQL sync_status y crear_alerta_automatica (HU-03). • Alertas en tiempo real por WebSocket en el panel (HU-01). • Despliegue del backend en Render y del panel en Vercel. • Botón «Marcar como atendida» con trazabilidad (HU-06). • Consentimiento informado digital (HU-07), mapa Leaflet de sensores (HU-09) y filtros en el Historial (HU-10). • Modo oscuro e íconos Lucide. • Contenedores Docker para backend y frontend, y despliegue en AWS Academy Learner Lab sobre EC2 (HU-11).
Pruebas realizadas	• Prueba de WebSocket: 13 alertas recibidas en 5 minutos, cada una en menos de 1 segundo (registrada en la Ficha 15; script backend/test_websocket.py). • Chequeo de salud /api/health en Render: 5 de 5 componentes en línea. • Prueba de despliegue en EC2 (01/10): 8/8 sensores operativos, alertas activas y alerta en tiempo real recibida en el panel. • Los casos de prueba formales PU, PI, PR y PS (Guía 1.5, Tablas 14 y 15) están pendientes.
Evidencias	• Commits 5a899de y d4fd9c7 (21/09): ntfy y tópico seudonimizado. • Commits 241ac4c y 879a6e0 (23/09): conexión a Supabase y triggers (fichas 10 y 11). • Commits a298b91 y 0edf181 (24-25/09): WebSocket en backend y frontend (ficha 15). • Commits 588c6e8, 21e68ea, 7158cb9 y f969801 (25/09): ajustes para desplegar en Render. • Commits 2a37c2c y d3d25b4 (26/09): CORS para Vercel y botón «Marcar como atendida». • Commits dbba8bb, a1a0d42, 5cb0784, 998a37a y 55e09ca (27/09): modo oscuro, íconos, filtros, consentimiento y mapa.

Campo	Contenido
	• Commits fac32bb y f292da7 (01/10): Docker y CORS configurable. • Capturas aws_swagger.png, aws_panel_1.png, aws_panel_2.png y aws_panel_datos.png (01/10): DomoVida corriendo en EC2.
Problemas encontrados	• Neon (PostgreSQL) no funcionó: el proveedor de internet bloquea las conexiones SSL al puerto 5432. • Varios intentos fallidos de despliegue en Render (compilación de dependencias, versión de Python). • El panel en Vercel no podía consultar la API por la política CORS. • Riesgo identificado: mantener el repositorio dentro de una carpeta sincronizada con OneDrive puede generar copias en conflicto y bloquear archivos de git y de dependencias (node_modules) durante la sincronización. • Docker: el docker-compose.yml quedó en la carpeta equivocada, la imagen usó una base SQLite antigua y CORS no aceptaba el panel en Docker ni la IP de EC2. • No se pudo mantener la bitácora en Obsidian con la cuenta institucional.
Acciones tomadas	• Cambio de Neon a Supabase con Session Pooler IPv4, puerto 6543 (23/09), documentado en la ficha 18. • Requisitos sin versiones fijas, Python 3.11 forzado y Procfile en Render (25/09). • URL de Vercel agregada a CORS (26/09). • Acción preventiva: por recomendación, el repositorio se trasladó a la carpeta Documentos, fuera de OneDrive, y GitHub quedó como única fuente oficial del código; la copia antigua en OneDrive no se utiliza. • Docker: compose movido a la raíz, base SQLite propia en un volumen y CORS configurable por variable de entorno (01/10). • Registro reconstruido desde el historial de commits (este documento).
Pendientes	• HU-08: el código publicado guardaba el RUT con base64 truncado; la corrección a SHA-256 con sal estaba hecha en local. Resuelto el 02/10: subida en el commit 9b07078 y verificada en el caso PU-03. • Error del panel: mostraba «API desconectada» aunque la API respondía (el hook useDomovida no entregaba el valor conectado). Resuelto el 02/10 en el commit 9b07078, que además corrigió los despliegues de Vercel. • Ejecución de los casos de prueba formales PU, PI, PR y PS. Avance al 02/10: PU-03, PI-02 y PI-03 aprobadas, PR-01 parcial (registro en docs/fase2/pruebas/registro_pruebas.md); PS-01 pendiente por falta de autenticación en la API. • HU-12 (Escala SUS) → Sprint 6, según lo planificado.

Retrospectiva del Sprint 3 (parcial, al 1 de octubre)

Campo	Contenido
Verificación de la acción anterior	Acción del Sprint 2: usar prefijos feat/fix/docs en los commits. Se cumplió en la mayoría de los commits del Sprint 3 (por ejemplo, 5a899de feat, 2a37c2c fix, 98e0306 docs). Efecto: el historial permite reconstruir qué se hizo y cuándo, lo

Campo	Contenido
	que hizo posible este documento.
¿Qué funcionó bien?	Ritmo de avance alto (cerca de 50 commits en dos semanas); el sistema completo quedó desplegado en servicios gratuitos y luego en AWS; las decisiones técnicas quedaron documentadas (ficha 18).
¿Qué problemas tuvimos?	Muchos arreglos seguidos al desplegar (varios commits «fix» el mismo día), señal de que se subían cambios sin probarlos antes en un entorno igual al de producción; cambios importantes quedaron sin subir a GitHub (Consentimiento.tsx); no hubo bitácora.
¿Qué vamos a cambiar?	Verificar cada despliegue después de subir cambios y mantener activos los servicios gratuitos, porque los despliegues de Vercel fallaron siete veces seguidas durante cinco días sin detectarse y Supabase se pausó por inactividad; registrar la retrospectiva en el repositorio (carpeta agil/).
Acción concreta, responsable y fecha	Después de cada git push, revisar que Vercel y Render queden con el despliegue en «Listo/Live», y abrir el panel publicado al menos una vez por semana para que Supabase no se pause · Andrés Vergara · desde el 2 de octubre de 2026.

Sprint 3 (S7-S9) — actualización al 6 de octubre

Ficha de sprint (2 al 6 de octubre de 2026)

Tabla E. Ficha del Sprint 3, actualización del 2 al 6 de octubre

Campo	Contenido
Sprint o ciclo	Sprint 3 · actualización del 2 al 6 de octubre de 2026 (el sprint cierra el 11 de octubre)
Objetivo	Cerrar las validaciones pendientes del prototipo (rendimiento, continuidad sin conexión, entrega de notificaciones, interoperabilidad y seguridad) y dejar el repositorio listo para la entrega de la Guía 2.4.
Historias comprometidas	HU-02, HU-03, HU-06 y HU-08 (validación formal pendiente al corte del 01/10), más dos tareas técnicas surgidas de las pruebas: seguridad de la API y recurso HL7 FHIR Patient.
Historias terminadas	HU-08 terminada: RUT seudonimizado (PU-03) y RLS en eventos, alertas y sensores (PI-03 y PI-03b). HU-02 validada: 30 de 30 avisos ntty recibidos en el celular (PR-02). HU-06 reforzada: atender una alerta exige la sesión del cuidador y registra su correo en resuelto_por (PS-01b). HU-03 reabierta: sin internet se guardaron y mostraron 10 de 10 caídas (PR-03), pero no se cumple el criterio de sincronizar el 100 % con Supabase al reconectar. Pasa al Sprint 4.
Funcionalidades implementadas	• Envío por WebSocket antes que ntty y hora de Chile en los avisos (-17 % de latencia). • HL7 FHIR R4: Observation y Bundle (02/10) y Patient seudonimizado con búsqueda por paciente (05/10). • Regla de la puerta principal en el backend: alerta de noche (22:00-07:00) o si queda abierta 10 minutos o más. • Seguridad: clave de API para los sensores (cabecera X-API-Key) e inicio de sesión del cuidador con Supabase Auth; RLS en la tabla sensores. • Panel: componentes simulados identificados, base de datos en uso y hora de atención de cada alerta. • Supabase: search_path fijo en dos funciones y eliminación de dos índices duplicados. • README y .env.example reescritos según la guía de apoyo del proyecto.
Pruebas realizadas	• PU-03, PI-02, PI-03 y PI-04 aprobadas (02/10). • PR-01 y PR-01b: 30/30 alertas por WebSocket; 1.577 ms en Render + Supabase y 216 ms en AWS EC2 (02/10). • PR-02: 30/30 notificaciones en el celular; PR-03: 10/10 caídas sin internet (03/10). • PU-04: 10/10; PI-05: 8/8; PS-01: 7/7 unitaria y 3/3 en producción; PS-01b: 7/7 unitaria y 3/3 en producción; PI-03b: 0 filas para el rol anónimo (05/10).
Evidencias	• Commits 1c8c203, 1da626e y 8a15d5e (02/10): ajuste de latencia, FHIR y pruebas iniciales.

Campo	Contenido
	- Commits 032f475 y b34bda0 (03/10): pruebas PR-02 y PR-03. - Commits bc31fb5, d7a2c88, 1688327, b23df2f, 0f01d89 y dd5d23c (05/10): panel, regla de la puerta, seguridad y FHIR Patient. - Commits e7a85d5, 52f2f36 y 8338d7b (05-06/10): README, Guía 2.4 y correcciones en Supabase. - Registro de pruebas y capturas en docs/fase2/evidencias_proyecto/documentacion.
Problemas encontrados	- Oracle Cloud Santiago no tenía capacidad para las máquinas gratuitas (03 y 05/10). - navigator.onLine no detecta el corte de internet en Windows, lo que invalidó una corrida de PR-03. - Al activar la clave de API, la clave guardada no coincidía con la del servidor (40 frente a 43 caracteres) y una clave quedó expuesta durante la depuración; el portapapeles de PowerShell no funcionó de forma confiable. - El Security Advisor de Supabase detectó la tabla sensores sin RLS, dos funciones con search_path mutable y un índice duplicado.
Acciones tomadas	- Despliegue en Oracle postergado; se mantiene AWS EC2 como entorno de comparación. - El script de PR-03 comprueba el corte con una consulta a ntfy.sh en lugar de navigator.onLine. - Diagnóstico con una huella SHA-256 parcial publicada en /api/health, clave expuesta descartada y clave nueva generada directamente en un archivo fuera del repositorio. - RLS activado en sensores el mismo día (PI-03b); search_path e índices corregidos el 06/10.
Pendientes	- HU-03: sincronización de los eventos del modo borde con Supabase al volver la conexión → Sprint 4. - Protección de las consultas de lectura de la API → Sprint 4. - Despliegue en Oracle Cloud Santiago → cuando haya capacidad. - PR-04 (carga con Locust) → Sprint 5; HU-12 (Escala SUS) → Sprint 6. - Matriz de trazabilidad, Product Vision, diagramas UML y manual técnico según la guía de apoyo del proyecto.

Retrospectiva del Sprint 3 (actualización al 6 de octubre)

Campo	Contenido
Verificación de la acción anterior	Acción del Sprint 3 (parcial): revisar Vercel y Render después de cada git push. Se cumplió: después de cada push se verificó /api/health y el panel publicado, lo que permitió detectar en minutos que la clave de API no coincidía. Efecto: ningún despliegue fallido quedó sin detectar.
¿Qué funcionó bien?	Las pruebas se ejecutaron contra el sistema publicado con scripts versionados, y los resultados quedaron registrados el mismo día. Comparar huellas SHA-256 permitió diagnosticar la clave sin mostrarla.

Campo	Contenido
¿Qué problemas tuvimos?	Tiempo perdido al copiar y pegar claves entre aplicaciones, con una clave expuesta; la sincronización del modo borde no estaba implementada aunque HU-03 figuraba como terminada; dependencia de la capacidad de Oracle.
¿Qué vamos a cambiar?	Generar los secretos directamente en archivos fuera del repositorio, sin pegarlos en chats ni consolas, y revisar el Security Advisor de Supabase al cierre de cada sprint. Marcar una historia como terminada solo cuando su prueba cumpla todos los criterios de aceptación.
Acción concreta, responsable y fecha	Al cierre de cada sprint, ejecutar el Security Advisor de Supabase y contrastar el estado de cada historia con el registro de pruebas · Andrés Vergara · desde el cierre del Sprint 3 (11 de octubre de 2026).